

Reviewer Pack — Minimal Rank of Quadratic Forms vs Boolean Tensor Product Va...

Entry c3063a12e8cb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 07:30:33 UTC

Minimal Rank of Quadratic Forms vs Boolean Tensor Product Valuations

Entry ID: c3063a12e8cb **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 07:30:33 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Geometry (Quadratic Forms) **Field B** (complexity object): Communication Complexity (Boolean Tensor Product)

Statement:

[‘For every Boolean matrix $M \in \{0,1\}^{n \times n}$ with entries x_{ij} representing the presence of a connection between variables i and j in a communication complexity scenario, the minimal rank of the quadratic form corresponding to the tensor product valuation of M is $\Theta(\log n)$.’, ‘Equivalently, for any instance of communication complexity where the Boolean matrix M can be represented as the tensor product valuation of some underlying structure, there exists a quadratic form F with minimal rank such that its associated symmetric bilinear form $B_F(x) = \sum_{i,j} x_i x_j F(i,j)$ captures the essential structure of the communication process.’, ‘The minimal rank of F is monotonically related to the complexity of the communication problem, providing an algebraic measure for the underlying boolean tensor product structure.’]

Rationale (proposer’s reasoning):

[‘Quadratic forms provide a rich algebraic tool that can potentially capture the complexity of certain communication processes by encoding the symmetries and interactions within the boolean tensor product valuation.’, ‘This conjecture aims to establish a connection between the geometric properties of quadratic forms and the computational aspects of communication complexity, which could lead to new insights into both fields.’, ‘The minimal rank of a quadratic form has been used in other contexts to measure the complexity of algebraic problems, suggesting its applicability to understanding communication processes.’]

Taxonomy category: quadratic_form_boolean_tensor_product_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a1b7aa4ec29c2da6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all generated Boolean matrices M , the minimal rank of the corresponding quadratic form F is within a factor of 2 from $\log n$ with a Spearman's rank correlation coefficient (ρ) ≥ 0.8 across 30 random seeds. The conjecture is falsified if any seed produces a minimal rank of F that deviates by more than a factor of 2 from $\log n$ or $\rho < 0.8$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - intitle:Minimal Rank Quadratic Forms AND Boolean Tensor Product Valuations - algebraic geometry quadratic forms AND communication complexity Boolean Tensor Product - tensor product valuation Boolean matrix AND minimal rank quadratic form

Top relevant hits considered: - [http://arxiv.org/abs/0904.1347v4] The product on smooth and generalized valuations - [http://arxiv.org/abs/1008.3482v1] Product numerical range in a space with tensor product structure - [http://arxiv.org/abs/1709.08394v7] Contravariant Form on Tensor Product of Highest Weight Modules - [http://arxiv.org/abs/1912.00347v3] Equiresidual algebraic geometry I: The affine theory - [http://arxiv.org/abs/2204.10334v1] Machine Learning Algebraic Geometry for Physics - [http://arxiv.org/abs/1305.0635v3] Quantum group-twisted tensor products of C^* -algebras - [http://arxiv.org/abs/1211.0398v1] Extending valuations to formal completions - [http://arxiv.org/abs/1607.03608v3] On the tensor product of linear sites and Grothendieck categories

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        # Find pivot
        max_row = i
        for k in range(i+1, n):
            if abs(A[k][i]) > abs(A[max_row][i]):
                max_row = k
```

```

    A[i], A[max_row] = A[max_row], A[i]

    # Eliminate below
    for k in range(i+1, n):
        factor = Fraction(A[k][i], A[i][i])
        for j in range(i, n):
            A[k][j] -= factor * A[i][j]

    # Back substitution
    x = [0]*n
    for i in range(n-1, -1, -1):
        x[i] = Fraction(A[i][-1], A[i][i])
        for k in range(i-1, -1, -1):
            A[k][-1] -= A[k][i] * x[i]

    return [x[i] for i in range(n)]

def minimal_rank(F):
    n = len(F)
    A = [[F[i][j] - F[i][k] * F[k][j] for j in range(n)] for k in range(1, n)]
    rank = 0
    for row in gaussian_elimination(A):
        if any(row):
            rank += 1
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    M = [[random.choice([0, 1]) for _ in range(n)] for _ in range(n)]

    F = [[0]*n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if M[i][j]:
                F[i][j] = 1

    min_rank_F = minimal_rank(F)
    expected_min_rank = math.log2(n)

    return {
        "metric_name": "minimal_rank",
        "metric_value": min_rank_F,
        "instances_tested": n * n,
        "conjecture_holds": abs(min_rank_F - expected_min_rank) <= 0.5 * expected_min_rank,
        "counterexample": "" if conjecture_holds else f"n={n}, M={M}, F={F}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

```

```

results.append(result)

mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
std_metric_value = (sum((r["metric_value"] - mean_metric_value)**2 for r in results) / len(results))**0.5
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{results[0]['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print(f"RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fc0eb36b.py", line 80, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fc0eb36b.py", line 63, in run_trial
    min_rank_F = minimal_rank(F)
                  ^^^^^^^^^^^^^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fc0eb36b.py", line 45, in minimal_rank
    A = [[F[i][j] - F[i][k] * F[k][j] for j in range(n)] for k in range(1, n)]
          ^

```

NameError: name 'i' is not defined. Did you mean: 'id'?

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11672
2	propose	ollama_remote	glm4:latest	0	0	11972
3	propose	ollama_remote	glm4:latest	0	0	11371
4	preregistration	ollama_remote	glm4:latest	0	0	6121
5	novelty	ollama_remote	glm4:latest	0	0	4584
6	novelty	ollama_remote	glm4:latest	0	0	5994
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	29682
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14011
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10070
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9697
11	judge	ollama_remote	glm4:latest	0	0	8181

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 123357 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in *research/audit/c3063a12e8cb.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/c3063a12e8cb.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/c3063a12e8cb.tar.gz* (if generated)